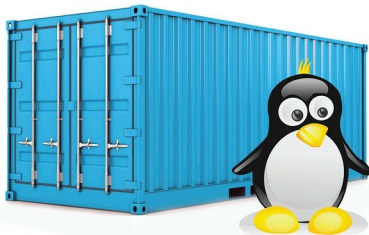


Linux Containers are Here to Stay!



The historic rise in the adoption of Linux containers makes for interesting reading.

Steve, a software engineer with a big tech firm in Silicon Valley, has been working with his team on a cool new artificial intelligence system that the team has been developing locally. The team members have built it over a year, using a collection of open source tools, libraries and software, and are now faced with the mammoth task of deploying it onto live servers—the stage at which the real problems start kicking in!

The team had built all the scripts using Python 2.7, before realising that the production servers had only Python 3 installed. There were a number of dependencies that were inadvertently upgraded and, as a result, a lot of the code got deprecated. There were varied security policies, network topologies, and different types of storage that could not be simultaneously handled by the application. Pipelines broke down, responses were lost and ultimately, the app crashed, splattering error messages across the terminal.

There are obvious issues with deployment, one being the lack of compatibility on multiple platforms and operating systems. There are system paths that need to be set, dependencies

that need to be installed, and versions to be monitored for multiple packages. Of course, modern-day package managers, virtual environments, and automated testing do make things that much easier. Still, getting the infrastructure to production-grade from a local development server is a nightmarish task for even the most skilled team of developers. This situation was extremely common for most developers until the inception of isolated environments within a Linux operating system—an idea intuitively named Linux containers.

The idea was not the first of its kind—FreeBSD jails and their *chrooted* equivalents on Linux did exist since 2000. However, eventually, support

grew around Linux containers, initiated by Jacques Gélinas' VServer project. The containers we see today are a technology that allows you to isolate applications as part of a package—including the entire runtime environment that it depends on for execution. This makes it easy to move the 'contained' application from development to testing and production while retaining full functionality. Linux containers also help minimise conflicts that may arise between development and operations teams by segregating areas of responsibility. Developers focus on apps, and the operations team focuses on infrastructure. A container

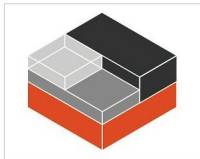


Figure 1: LXC: The Linux container



Figure 2: From development to production: The warrior's path (courtesy: commitstrip.com)